Università degli Studi di Palermo

dipartimento di ingegneria unipa

Strutture di controllo del flusso del programma C

CALCOLATORI ELETTRONICI – FONDAMENTI DI PROGRAMMAZIONE

a.a. 2025/2026

Prof. Roberto Pirrone

CHILAB

Espressioni logiche

- Le strutture di selezione Si basano sulla valutazione di *espressioni booleane o logiche*
  - Espressioni il cui valore è *vero* o *falso*
- In C:
  - Falso → l'espressione vale 0
  - Vero → l'espressione vale 1 (o comunque diverso da 0)

Espressioni logiche

- Le espressioni booleane sono costruite a partire da due tipologie di operatori:

  - Operatori relazionali
    Stabiliscono la verità di un confronto tra due sotto-espressioni

  - Operatori logici
    Connettono tra loro più sotto-espressioni secondo la logica classica

Espressioni logiche

- Operatori relazionali

Operator	Meaning
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to
==	equal to
!=	not equal to

Espressioni logiche

- Operatori logici

TABLE 4.3 The && Operator (and)

operand1	operand2	operand1 && operand2
nonzero (true)	nonzero (true)	1 (true)
nonzero (true)	0 (false)	0 (false)
0 (false)	nonzero (true)	0 (false)
0 (false)	0 (false)	0 (false)

Espressioni logiche

- Operatori logici

TABLE 4.4 The || Operator (or)

operand1	operand2	operand1 || operand2
nonzero (true)	nonzero (true)	1 (true)
nonzero (true)	0 (false)	1 (true)
0 (false)	nonzero (true)	1 (true)
0 (false)	0 (false)	0 (false)

Espressioni logiche

- Operatori logici

TABLE 4.5 The ! Operator (not) *unario*

operand1	!operand1
nonzero (true)	0 (false)
0 (false)	1 (true)

Espressioni logiche

- Precedenza degli operatori

È possibile inserire le parentesi anche per modificare la priorità dell'applicazione degli operatori nelle espressioni logiche.

Operator
function calls
! + - & (unary operators)
* / %
+ -
< <= >= >
== !=
&&
||
=

Espressioni logiche

x	y	z	flag
3.0	4.0	2.0	0

```
1.  !flag
2.  x + y / z  <=  3.5
3.  !flag || (y + z  >=  x - z)
4.  !(flag || (y + z  >=  x - z))
```

Espressioni logiche

x	y	z	flag
3.0	4.0	2.0	0

```
1.  !flag                          /* !0 is 1 (true)          */
2.  x + y / z  <=  3.5             /* 5.0 <= 3.5 is 0 (false) */
3.  !flag || (y + z  >=  x - z)    /* 1 || 1 is 1 (true)      */
4.  !(flag || (y + z  >=  x - z))  /* !(0 || 1) is 0 (false)  */
```

Espressioni logiche

- Valutazione *corto circuito*

- Le espressioni logiche vengono valutate da *sinistra a destra* (a meno di alterazioni dovute alle parentesi)

- *La valutazione si ferma non appena la verità dell'intera espressione è accertata*

Espressioni logiche

- Valutazione *corto circuito*

`a && b` è immediatamente falso se `a` è falso *<u>senza valutare</u>* `b`

`a || b` è immediatamente vero se `a` è vero *<u>senza valutare</u>* `b`

Espressioni logiche

- Appartenenza di una variabile ad un range

```
min <= x  &&  x <= max
```

Espressioni logiche

- Appartenenza di una variabile ad un range

```
min <= x  &&  x <= max
```

- Confronto tra caratteri

• `'9' >= '0'`	vero
• `'a' < 'e'`	vero
• `'B' <= 'A'`	falso
• `'Z' == 'z'`	falso
• `'a' <= 'A'`	falso
• `'a'<=ch && ch<='z'`	vero se `ch` è una lettera minuscola (è un intervallo)

Espressioni logiche

- Il risultato di un'espressione logica può essere assegnato ad una variabile

```
int  age;             /* input - a person's age              */
char gender;          /* input - a person's gender           */
int  senior_citizen;  /* logical - indicates senior status   */

senior_citizen = 1;   /* Set senior status */
scanf("%d", &age);                        /* Read the person's age */
senior_citizen = (age >= 65);             /* Set senior status */
```

Espressioni logiche del genere sono valide:

```
!senior_citizen

senior_citizen  &&  gender == 'M'
```

Espressioni logiche

- Come negare un'espressione logica semplice del tipo:

exp = *op1* <relazionale> *op2* ?

- !*exp*

- *op1* <relazionale_complementare> *op2*
  - > diventa <=
  - < diventa >=
  - == diventa !=

Espressioni logiche

- Come negare un'espressione logica complessa (che usa `&&`, `||` e `!`)?
- Teorema di De Morgan
- `!(`*exp1* `&&` *exp2*`)` = *exp1_comp* `||` *exp2_comp*
- `!(`*exp1* `||` *exp2*`)` = *exp1_comp* `&&` *exp2_comp*

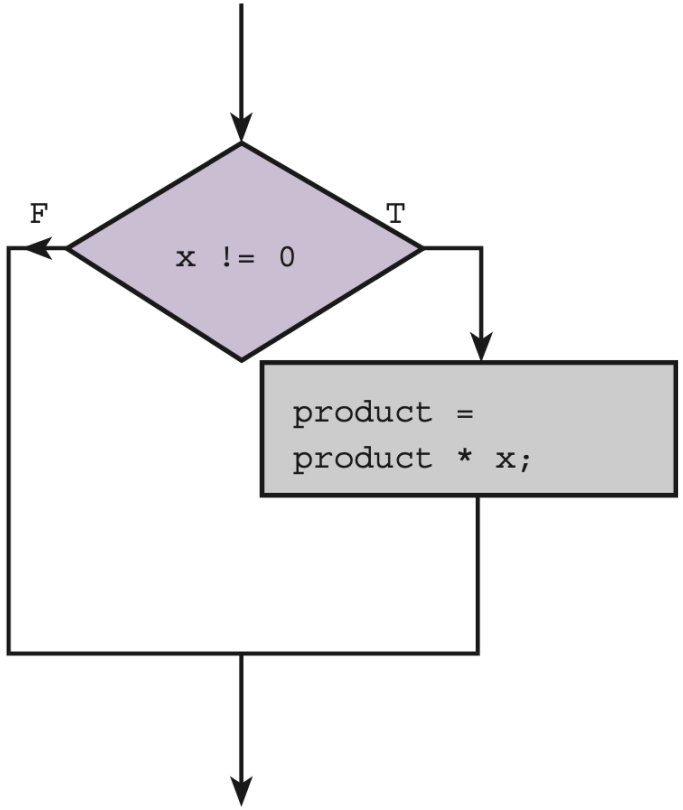
```
age > 25  &&  (status == 'S' || status == 'D')

age <= 25 || (status != 'S'  &&  status != 'D')
```

Strutture di selezione

- Selezione ad una alternativa

FORM: `if (`*condition*`)`
statement$_T$`;`

EXAMPLE:
```
if (x > 0.0)
    pos_prod = pos_prod * x;
```



Strutture di selezione

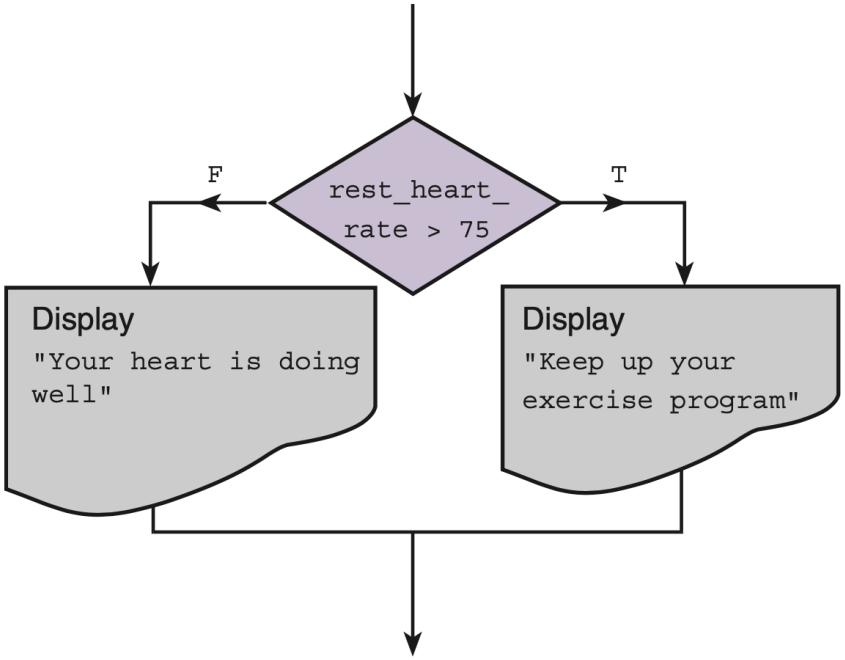
- Selezione a due alternative

FORM:

```
if (condition)
    statement_T;
else
    statement_F;
```

EXAMPLE:

```
if (x >= 0.0)
    printf("positive\n");
else
    printf("negative\n");
```



Strutture di selezione

- Selezione a più alternative

SYNTAX: `if (`*condition*$_1$`)`
statement$_1$
`else if (`*condition*$_2$`)`

```
if (x > 0)
      num_pos = num_pos + 1;
else
      if (x < 0)
            num_neg = num_neg + 1;
      else /* x equals 0 */
            num_zero = num_zero + 1;
```

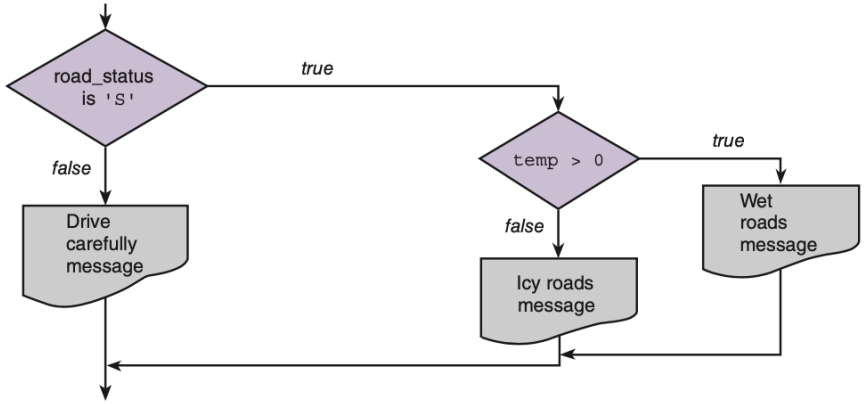
E

```
            num_neg = num_neg + 1;
      else /* x equals 0 */
            num_zero = num_zero + 1;
```

Strutture di selezione

- Selezione tra più alternative

```
if (road_status == 'S')
      if (temp > 0) {
            printf("Wet roads ahead\n");
            printf("Stopping time doubled\n");
      } else {
            printf("Icy roads ahead\n");
            printf("Stopping time quadrupled\n");
      }
else
      printf("Drive carefully!\n");
```



Strutture di selezione: `switch`

SYNTAX: `switch` (*controlling expression*) {

label set$_1$

statements$_1$

`break;`

label set$_2$

statements$_2$

`break;`

. . .

label set$_n$

statements$_n$

`break;`

`default:`

statements$_d$

}

EXAMPLE:

```
/* Determine life expectancy in hours of an
   incandescent light bulb */
switch (watts) {
case 25:
      life = 2500;
      break;

case 40:
case 60:
      life = 1000;
      break;

case 75:
case 100:
      life = 750;
      break;

default:
      life = 0;
}
```

Solo di tipo `int` o `char`

Interrompe l'esecuzione

Caso standard, quando l'espressione di controllo non è uguale a nessuna etichetta

Iterazioni

- Una iterazione o ciclo è controllato tramite una espressione logica e consta sempre di tre componenti:
  - Una istruzione di inizializzazione della condizione di controllo
  - La vera e propria condizione che itera il ciclo se è vera
  - Una istruzione di *aggiornamento* della verità della condizione per terminare il ciclo

Iterazioni

- Sintassi delle strutture di controllo iterative

```
for (espr. di inizializzazione ;
        espr. di controllo ;
        espr. di aggiornamento )
                istruzioni da iterare ;
```

Iterazioni

- Sintassi delle strutture di controllo iterative

```
for (i = 0; i < max; i += 1)
    printf("%d\n", i);
```

Cosa fa questo codice?

Iterazioni

- Sintassi delle strutture di controllo iterative

espr. di inizializzazione ;

`while` (*espr. di controllo*)

{

istruzioni da iterare ;

espr. di aggiornamento ;

}

Iterazioni

- Sintassi delle strutture di controllo iterative

```
scanf("%d", &n);
ev = 0;
while (ev < n) {
    printf("%3d", ev);
    ev = ev + 2;
}
printf("\n");
```

Cosa fa questo codice?

Riscriviamolo con il for !!

Iterazioni

- Sintassi delle strutture di controllo iterative

```
do
{
        istruzioni da iterare ;
        espr. di inizializzazione / aggiornamento ;
                        /* l'inizializzazione è parte
                        delle istruzioni da iterare */

}
while (espr. di controllo)
```

Iterazioni

- Sintassi delle strutture di controllo iterative

```
do
      scanf("%c", &skip_ch);
while (skip_ch != '\n');
```

Cosa fa questo codice?

Iterazioni

- Sintassi delle strutture di controllo iterative

```
do
      scanf("%c", &skip_ch);
while (skip_ch != '\n');
```